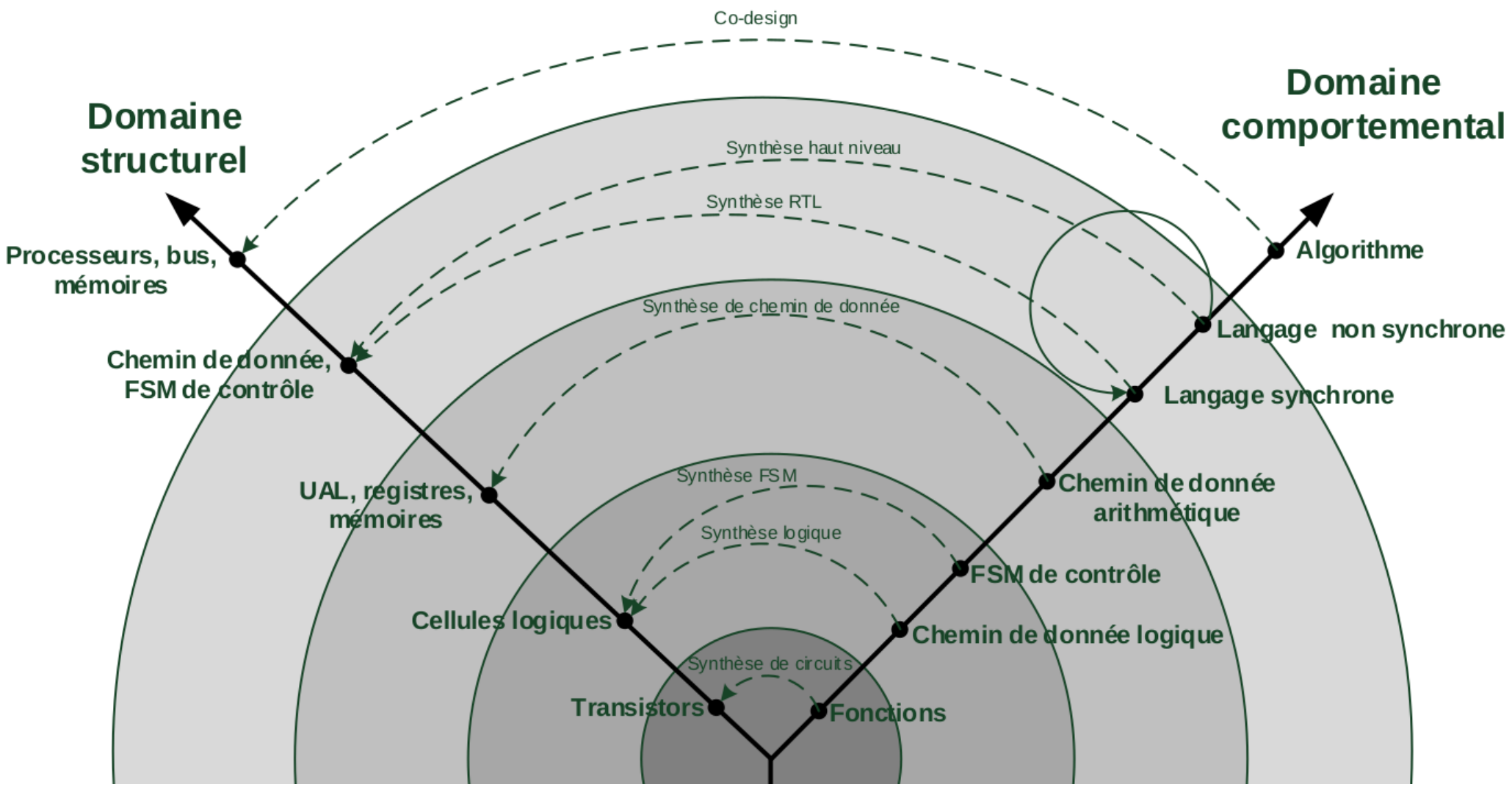
DÉPARTEMENT de PHYSIQUE

SYNTHÈSE RTL

Dr. Salaheddine H. Sfar

slaheddine.sfar@isima.rnu.tn


Co-design
Domaine structurel
Domaine comportemental
Synthèse haut niveau
Synthèse RTL
Synthèse de chemin de donnée
Synthèse FSM
Synthèse logique
Synthèse de circuits
Processeurs, bus, mémoires
Chemin de donnée, FSM de contrôle
UAL, registres, mémoires
Cellules logiques
Transistors
Algorithme
Langage non synchrone
Langage synchrone
Chemin de donnée arithmétique
FSM de contrôle
Chemin de donnée logique
Fonctions

RAPPEL-SYSTÈMES RÉACTIFS ET TEMPS RÉEL

- Système réactif : Un système qui a une interaction permanente avec l'environnement
  - Souvent, ces systèmes sont également des systèmes en temps réel
- Système Temps Réel : Un système qui doit satisfaire des contraintes temporelles
  - Temps réel à contraintes strictes "*hard real-time*"
    - Le respect des contraintes temporelles est indispensable au bon fonctionnement.
    - Le non-respect conduit à
      - des résultats inexploitables
      - des risques pour la mission, le système et/ou son environnement
  - Temps réel à contraintes relatives "*soft real-time*"
    - Le non-respect des contraintes temporelles ne conduit pas à un mauvais fonctionnement
    - Il induit juste certaines nuisances
      - coût supplémentaire
      - agacement des opérateurs (temps de réponse) imprécisions admissibles...
  - Temps réel à contraintes mixtes "*firm real-time*"

RAPPEL-SYSTÈMES RÉACTIFS ET TEMPS RÉEL

Fonctions à assurer

- Acquisition des données
- Contrôle séquentiel
- Boucle de contrôle
- Supervision
- Analyse des données
- Stockage des données
- Interface homme-machine

Exigences

- Contraintes temporelles : *timing constraints*

  ⇨ Déterminisme temporel : respect des échéances

- Parallélisme : *concurrency*
- Prévisibilité des comportements : *predictability*

  ⇨ Déterminisme logique : les mêmes données en entrées donnent les mêmes résultats en sortie,

- Sûreté de fonctionnement : *dependability*

RAPPEL-SYSTÈMES RÉACTIFS ET TEMPS RÉEL

Les principaux problèmes

- La plupart des systèmes réactifs temps réel sont faits de composants communiquant qui évoluent en concurrence.
- On retrouve donc les problèmes dus au parallélisme (aspects qualitatifs):
  - Communication
  - Synchronisation
  - Organisation du flot de calcul
- Il y a en plus les aspects quantitatifs, imposés par les contraintes temporelles, essentiellement liés à la vitesse d'exécution.

SYSTÈMES RÉACTIFS ET TEMPS RÉEL

Comment sont-ils programmés - modélisés?

Langage impératif séquentiel (C)+RTOS

- RTOS : *Real Time operating system*
- Approche la plus populaire, où beaucoup d'expérience existe
- Cependant
  - Il est difficile à comprendre, déboguer, vérifier et maintenir une application existante
  - Il y a peu de possibilités d'analyse automatique
  - Le RTOS peut introduire du non déterminisme

Machine à états finis

- Déterministe
- Peut être formellement vérifiée
- La coopération entre plusieurs FSM peut facilement devenir délicate
  - Ne supporte pas directement la hiérarchie et le parallélisme
  - Très sensible aux modifications de la spécification
- Seules les machines de petite taille sont humainement compréhensibles

SYSTÈMES RÉACTIFS ET TEMPS RÉEL

Comment sont-ils programmés - modélisés?

Langages de programmation concurrente

- ADA : norme ISO 8652 dernière version en 2012
  - https://github.com/jcolag/SiliconAda
- Occam : propriété de ST Microelectronics dernière version 2.1 (1994)
  - http://www.wotug.org/cpa2004/papers/339-peel.pdf
- Java
  - *Real-Time Specification for Java* (RTSJ)
  - https://www.rtsj.org/
  - Dernière version RTSJ 2.0 approuvée en 2019 par le Java Community Process (JCP)
- LabVIEW : propriété de National Instrument, pas de standard
  - LabVIEW FPGA

- Points quasi communs :
  - Piles d'exécution : thread processus ou tâches : concurrences
  - Synchronisation par : mutex , sémaphore (mémoire partagé) , rendez vous (échange de message) : déterminisme
- Essentiellement asynchrones
- Déterminisme limités : ordre d'exécution parfois incontrôlé
- Difficile à prévoir et à vérifier

APPROCHE SYNCHRONE

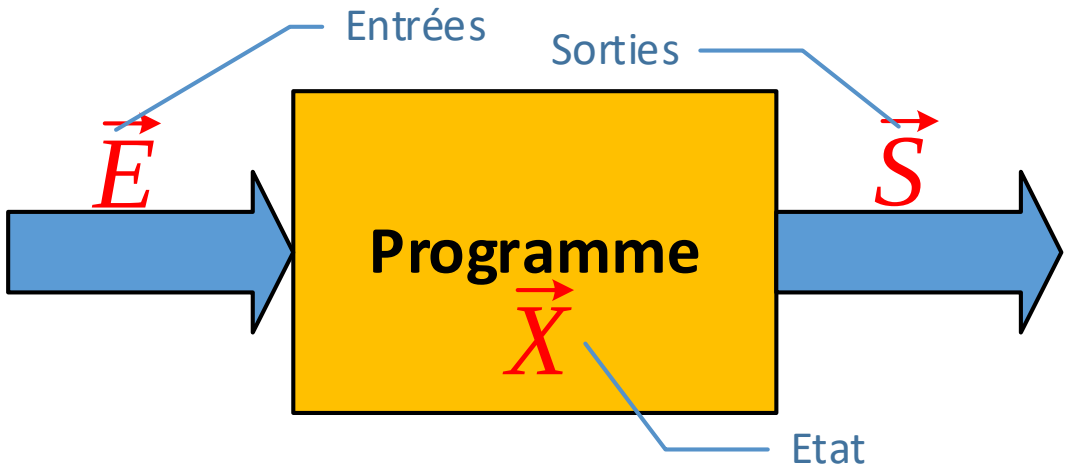
Sémantique

- La signification (sémantique) du programme P est donnée par la caractérisation de toutes les correspondances possibles entre les séquences d'entrées (E_1,E_2 , ...) et les séquences de sorties (S_1,S_2, ...)
- Un programme synchrone étant déterministe, la correspondance est fonctionnelle

$\overrightarrow{X'} = f(\vec{X}, \vec{E})$ Fonction état suivant

$\vec{S} = g(\vec{X}, \vec{E})$ Fonction de sortie

$\overrightarrow{X_0}$ Etat initial



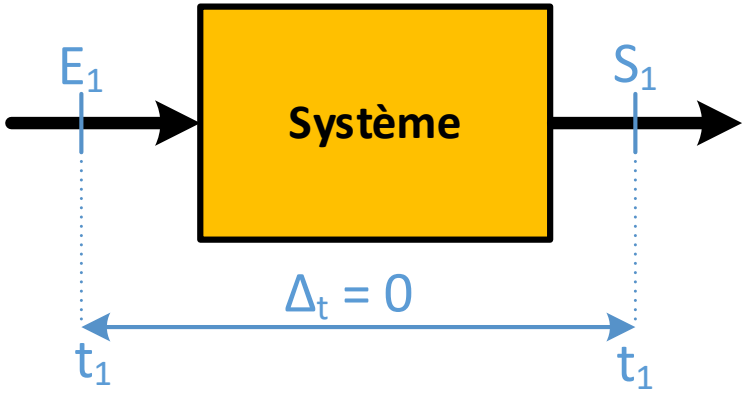
APPROCHE SYNCHRONE

Idée de base

- Les systèmes sont idéalisés
  - Les systèmes produisent leurs sorties de manière synchrone avec leurs entrées
- Simplification de la composition et de l'analyse

Hypothèse de synchronisation parfaite

- Les réactions sont instantanées de sorte que les activations et les productions de sorties sont synchrones
- C'est comme si les programmes étaient exécutés sur une machine infiniment rapide



APPROCHE SYNCHRONE

Idée de base

- Les systèmes sont idéalisés
  - Les systèmes produisent leurs sorties de manière synchrone avec leurs entrées
- Simplification de la composition et l'analyse

Les réactions sont atomiques

- Les réactions sont atomiques et n'interfèrent pas les unes avec les autres
- On peut voir la séquence d'événements d'entrée comme une séquence d'événements d'horloge qui déclenche l'exécution du système.

Seul l'ordre total des événements est défini.
Il n'y a pas de mappage de leurs durées dans le temps physique

LES LANGAGES SYNCHRONES

Principe

- Les langages synchrones sont des langages spécialisés, pas des langages d'usage général.
- Les langages synchrones ont été conçus spécialement pour les systèmes réactifs
- Ils sont à la fois
  - plus simples (systèmes idéalisés)
  - plus propres (sémantique à base de fonctions mathématiques)
- que les langages d'usage général
- Ils sont souvent plus efficaces à l'exécution (processus de compilation spécifique – hors du contexte de ce cours)

Exemples

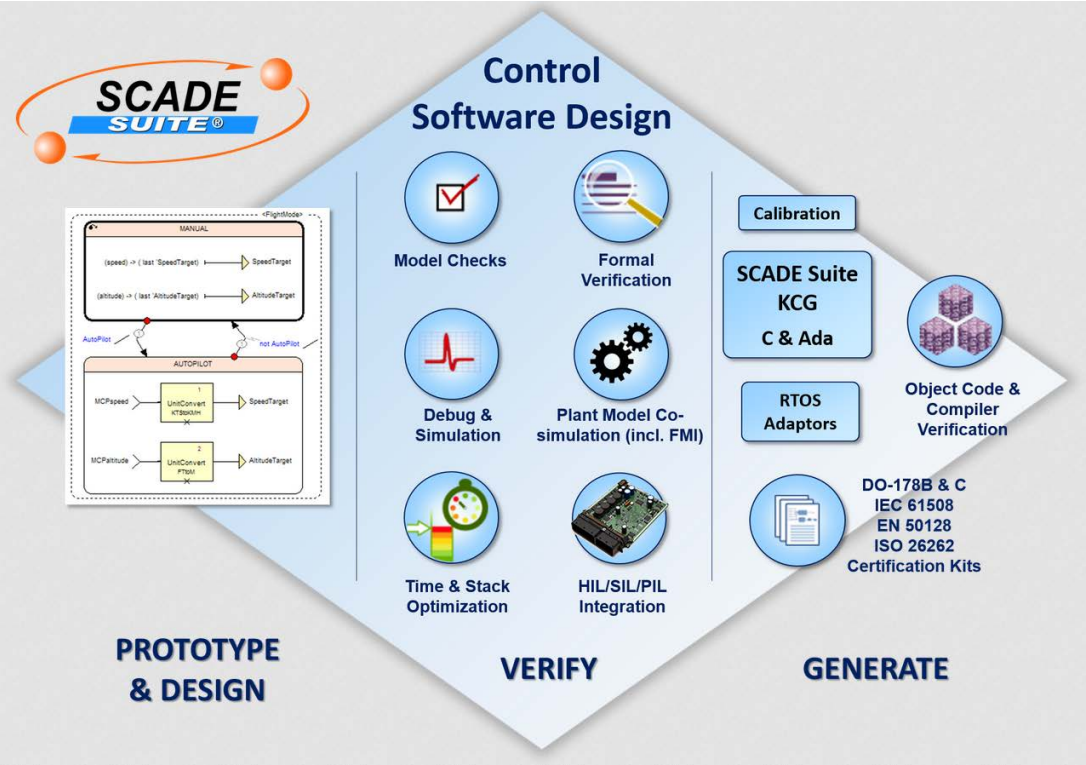
- Esterel
- Lustre
- Signal
- Statecharts

ESTEREL

- Le langage créé dans les année 80 avec l'association de de l'Ecole des Mines de Paris et l'Institut national de recherche en informatique et en automatique (INRIA)
- plusieurs compilateurs sortis
  - Codes C
  - Une implémentation RTL (VHDL ou Verilog)
- Projet académique POLIS (Université de Berkeley)
  - https://ptolemy.berkeley.edu/projects/embedded/research/hsc/
- La version commerciale d'Esterel est l'environnement de développement Esterel Studio développé par Esterel Technologies
- Esterel Studio a été racheté en 2009 par Synfora aujourd'hui Synopsys
- La société fut absorbée par la société américaine Ansys en 2012.

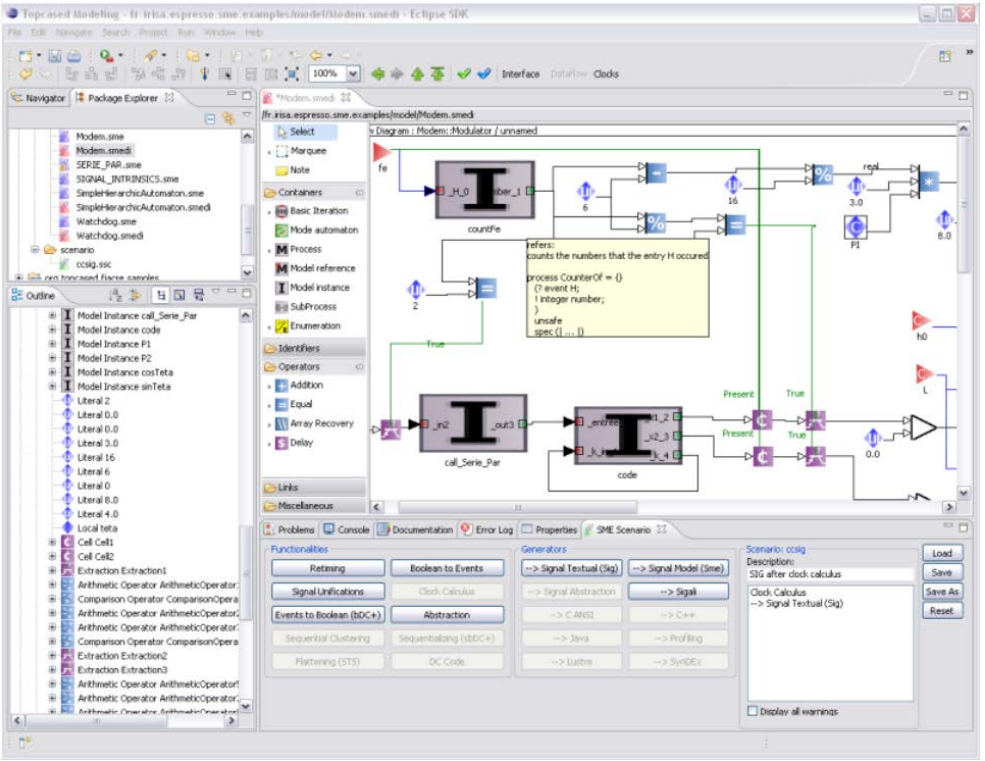
LUSTRE

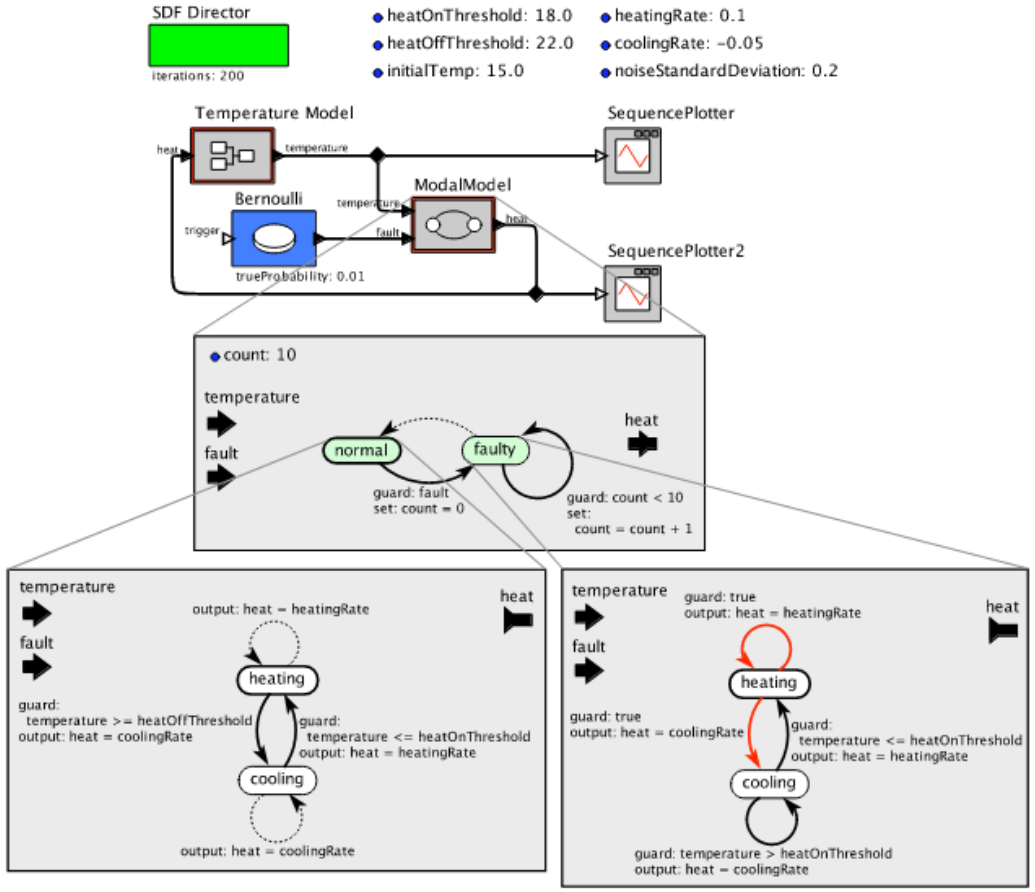
- Proche du langage Esterel
- Publié en 1991 (proceedings IEEE)
- Entre dans le monde industriel avec la société Esterel Technologies en 1993
  - Environnement de développement SCADE (*Safety Critical Application Development Environment*)
- SCADE est aujourd'hui un produit Ansys
  - https://www.ansys.com/products/embedded-software/ansys-scade-suite



SIGNAL

- SIGNAL a été principalement développé par une équipe de l'INRIA depuis les années 1980, en même temps que des langages de programmation similaires, Esterel et Lustre.
- L'ensemble d'outils Polychrony est un environnement de développement open source pour les systèmes critiques/embarqués basés sur SIGNAL.
- http://polychrony.inria.fr/



STATECHARTS

- Une variantes des diagrammes états transition
- Inventés par l'informaticien David Harel en 1987
- Appelé parfois Harel-Statecharts
- Une variante fait désormais partie du langage de modélisation unifié (UML)
- Le projet Ptolemy de l'université Californie étudie la modélisation, la simulation et la conception de systèmes embarqués simultanés en temps réel : supporte entre autre Satecharts
  - https://ptolemy.berkeley.edu/index.htm
- Il permet de générer du code C et aussi du VHDL RTL